

supports the features of C++11 and C++14. The latter has become the default standard for GCC 6. There were many changes made to the core language by the standard C++11. The inclusion of a new 64-bit integer data type, called *long long int*, is a change made in C++ by the C++11 standard. Earlier, C++ only had 32-bit integers called *long int*. External templates were also added to C++ by this standard.

Many more changes were made to the core of the C++ language by the C++11 standard, but the changes were not limited to the core alone — the C++ standard library was also enhanced by the C++11 standard. Changes were made to the C++ standard library in such a way that multiple threads could be created very easily. New methods for generating pseudo-random numbers were also provided by the C++11 standard. A uniform method for computing the return type of function objects was also included by the C++11 standard. Though a lot of changes have been made to the standard library in C++11, the one that concerns us the most is the inclusion of a new header file called `<regex>`.

Regular expressions in C++11

In C++, support for regular expressions is achieved by making changes to the standard library of C++. The header file called `<regex>` is added to the C++ standard library to support regular expressions. The header file `<regex>` is also available in C++14 and, hence, what we learn for C++11 also applies to C++14. There are some additions to the header file `<regex>` in C++14, which will be discussed later in this article. There are three functions provided by the header file `<regex>`. These are `regex_match()`, `regex_search()` and `regex_replace()`. The function `regex_match()` returns a match only if the match is found at the beginning of a string, whereas `regex_search()` searches the entire string for a match. The function `regex_replace()` not only finds a match, but it replaces the matched string with a replacement string. All these functions use a regular expression to denote the string to be matched.

Other than these three functions, the header file `<regex>` also defines a number of classes like `regex`, `wregex`, etc, and a few iterator types like `regex_iterator` and `regex_token_iterator`. But to simplify and shorten our discussion, I will only cover the class `regex` and the three functions, `regex_search()`, `regex_match()` and `regex_replace()`. I believe it is impossible to discuss all the features of the header file `<regex>` in a short article like this, but the topics I will cover are a good starting point for any serious C++ programmer to catch up with professional users of regular expressions. Now let us see how regular expressions are used in C++ with the help of a small C++ program.

A simple C++ program using regular expressions

The code below shows a C++ program called `regex1.cc`. I am sure you are all familiar with the `.cc` extension of C++

programs. This and all the other C++ programs and text files used in this article can be downloaded from opensourceforu.com/article_source_code/September17C++zip.

```
#include <iostream>
#include <regex>

using namespace std;
int main( )
{
    char str[ ] = "Open Source For You";
    regex pat("Source");
    if( regex_search(str,pat) )
    {
        cout << "Match Found\n";
    }
    else
    {
        cout<<"No Match Found\n";
    }
    return 0;
}
```

I'm assuming that the syntax of C is quite well known to readers, who will understand the simple C++ programs we discuss in this article, so no further skills are required. Now let us study and analyse the program. The first two lines `#include <iostream>` and `#include <regex>` include the two header files `<iostream>` and `<regex>`. The next line of code `using namespace std;` adds the `std namespace` to the program so that `cout`, `cin`, etc, can be used without the help of the scope resolution operator (`::`). The line `int main()` declares the only function in this program, the `main()` function.

This is one problem we face when programming languages like C++ or Java are used. You need to write a lot of code to set up the environment and get things moving. This is one reason why you should stick with languages like Perl or Python rather than C++ or Java if your whole aim is to process a text file. But if you are writing system software and want to analyse a system log file, then using regular expressions in C++ is a very good idea.

The next line of code `char str[ ] = "Open Source For You";` initialises a character array called `str[ ]` with a string in which we will search for a pattern. In this particular case, the character array is initialised with the string `Open Source For You`. If you wish to replace the line of code `char str[ ] = "Open Source For You"` with `string str = "Open Source For You";` and thereby use an object `str` of string class of C++ instead of a character array, the program will still work equally well. Remember that the `string` class of C++ is just an instance of the template class `basic_string`.

This modified program called `string.cc` is also available for downloading. On execution with the commands `g++ string.cc` and `./a.out`, the program `string.cc` will also behave